

Cụm 1 - Giới thiệu Software Architecture

Mục lục

Software Architecture Foundation	2
Vì sao có khoá này?	3
Đối tượng người đọc	3
Cấu trúc tài liệu	3
Cách đọc tài liệu	4
Người đọc lần đầu (chuyên sâu)	4
Người ôn thi (nhẹ)	4
Người tham chiếu nhanh (đã làm SA)	4
Triết lý biên soạn	4
Quy ước hiển thị	5
Bắt đầu từ đâu?	5
1.1 Tổng quan Cụm 1: Giới thiệu Software Architecture	6
Vì sao Cụm 1 cần đứng trước	6
Bốn bài trong cụm	6
Bài 1.2: Software Architecture là gì?	6
Bài 1.3: Mục tiêu và kết quả học tập	6
Bài 1.4: Lộ trình đọc tài liệu	6
Cách đọc Cụm 1	6
Kết nối với các cụm sau	7
Sai lầm cần tránh ngay từ đầu	7
1.2 Software Architecture là gì?	8
Một câu hỏi đơn giản, ba câu trả lời khác nhau	8
Góc nhìn 1: Định nghĩa hình thức (Bass-Clements-Kazman)	8
Góc nhìn 2: Định nghĩa thực dụng (Martin Fowler)	8
Góc nhìn 3: Định nghĩa operational (cái gì architect thực sự làm)	9
Tổng hợp: Định nghĩa “operational” của tài liệu này	9
Ba ví dụ minh hoạ	9
Ví dụ 1: Chọn dùng PostgreSQL hay MongoDB cho hệ thống mại điện tử	9
Ví dụ 2: Đặt tên biến userCount hay numUsers trong một function	9
Ví dụ 3: Dùng JWT hay session cookie cho authentication trong một SaaS B2B	10
“Architecture is design”, nhưng không phải mọi design đều architectural	10
Vì sao kiến trúc lại đắt khi sửa?	10
1. Cascade effect	10
2. Conway’s law	10
3. Sunk cost của technology lock-in	11

Một “architect” thực sự làm gì?	11
Tóm tắt	11
1.3 Mục tiêu và kết quả học tập	12
Vì sao cần nói rõ mục tiêu?	12
Mục tiêu khoá (Aims)	12
Mục tiêu 1: Nhận biết và lập luận về architectural decisions	12
Mục tiêu 2: Áp dụng SOLID và các nguyên lý thiết kế cấp module	12
Mục tiêu 3: Biết 9 architecture styles phổ biến	12
Mục tiêu 4: Đọc và viết documentation kiến trúc chuẩn	12
Mục tiêu 5: Áp dụng vào case study thực	13
Kết quả học tập (Learning Outcomes)	13
Không nằm trong scope	13
1. Domain-Driven Design (DDD)	14
2. CQRS, Event Sourcing, Saga, Outbox pattern	14
3. Hexagonal / Clean / Onion Architecture	14
4. Cloud-native specifics	14
5. Performance engineering chi tiết	14
6. Security architecture chi tiết	14
Lộ trình học tiếp sau khoá	14
Nếu bạn đi hướng Architect chuyên nghiệp	14
Nếu bạn đi hướng Tech Lead	15
Nếu bạn đi hướng Distributed Systems	15
Nếu bạn đi hướng Cloud Architect	15
Tóm tắt	15
1.4 Lộ trình đọc tài liệu	16
Sơ đồ phụ thuộc giữa các cụm	16
Ba lộ trình chính	16
Lộ trình A: Chuyên sâu (chuẩn, 30-50 giờ)	16
Lộ trình B: Ôn thi (nhANH, 5-8 giờ)	17
Lộ trình C: Tra cứu (on-demand)	17
Bốn shortcut paths theo focus area	17
Shortcut 1: “Tôi chỉ muốn hiểu microservices” (8-12 giờ)	17
Shortcut 2: “Tôi muốn dạy team cách document architecture” (4-6 giờ)	17
Shortcut 3: “Tôi muốn master SOLID và improve code quality team” (10-15 giờ)	18
Shortcut 4: “Tôi chuẩn bị interview Senior/Staff Engineer” (15-20 giờ)	18
Lưu ý khi học	18
Đừng “cày” lý thuyết một mình	18
Đừng ngại quay lại	18
Đừng bị quyến rũ bởi trends	18
Code thực sự là phần thiết yếu	18
Tiếp theo	18

Software Architecture Foundation

Chào mừng bạn đến với khoá **Software Architecture Foundation**, một bộ tài liệu mở Tiếng Việt về Kiến trúc Phần mềm ở mức nền tảng sau đại học. Tài liệu này không cố gắng thay thế các sách kinh điển như *Clean Architecture* của Robert C. Martin hay *Software Architecture in Practice* của Bass-Clements-

Kazman, mà đóng vai trò một người dẫn đường: giải thích trực giác trước khi đi vào định nghĩa hình thức, sắp xếp các khái niệm thành lộ trình để tiêu hoá, và minh hoạ bằng ví dụ thực tế đủ sát để bạn áp dụng vào dự án ngay.

Vì sao có khoá này?

Software Architecture là một trong những lĩnh vực mà người mới rất dễ bị ngợp. Mở một quyển sách bất kỳ, bạn sẽ thấy hàng chục thuật ngữ: SOLID, ATAM, 4+1 views, hexagonal, layered, microservices, event-driven, CQRS, saga, BFF, sidecar, service mesh... Mỗi thuật ngữ lại dẫn tới một họ kỹ thuật con. Người học hay bị mất phương hướng: không rõ thuật ngữ nào dùng cho tình huống nào, đâu là cốt lõi, đâu là biến thể, và quan trọng nhất, đâu là *cách suy nghĩ* của một software architect chứ không chỉ là *danh mục công cụ* mà họ dùng.

Khoá này được biên soạn để giải quyết đúng vấn đề đó. Toàn bộ nội dung được sắp xếp theo một mạch logic duy nhất:

1. **Bắt đầu bằng nguyên lý nhỏ** (Cụm 2: SOLID, Cohesion-Coupling) vì kiến trúc tốt luôn bắt đầu từ code tốt.
2. **Lùi lại để nhìn bức tranh lớn** (Cụm 3: Architectural Thinking) để hiểu vì sao kiến trúc khác thiết kế.
3. **Hiểu cái cần tối ưu** (Cụm 4: Quality Attributes) trước khi chọn cách tối ưu.
4. **Học các công cụ chuẩn** (Cụm 5-6: Architecture Styles), từ đơn giản tới phức tạp.
5. **Biết cách truyền đạt** (Cụm 7: Documenting), kiến trúc không tài liệu hoá là kiến trúc chết.
6. **Áp toàn bộ vào case thực** (Cụm 8: Case Studies) để xem khi nào dùng cái gì.

Đối tượng người đọc

Khoá này được viết với giả định bạn đã có:

- Tối thiểu 1-2 năm viết code production (bất kỳ ngôn ngữ nào, thuận lợi nhất là Java/C#/Python/TypeScript).
- Hiểu cơ bản về OOP (class, interface, inheritance), HTTP/REST, database SQL/NoSQL.
- Đã từng làm việc trong team từ 3 người trở lên (vì đó là lúc kiến trúc bắt đầu quan trọng).

Nếu bạn còn thiếu một trong các yếu tố trên, các bài đầu vẫn đọc được nhưng phần code minh hoạ có thể hơi khó. Đừng vội bỏ, hãy quay lại sau khi đã có kinh nghiệm.

Đối tượng phù hợp nhất:

- **Học viên cao học** ngành Khoa học Máy tính / Kỹ thuật Phần mềm cần một tài liệu Tiếng Việt mạch lạc để ôn cho môn Kiến trúc Phần mềm.
- **Lập trình viên 2-5 năm kinh nghiệm** đang chuẩn bị bước lên vai trò Tech Lead, Senior Engineer, hoặc Architect.
- **Người tự học** muốn lấp khoảng trống giữa “biết viết code” và “biết thiết kế hệ thống”.

Cấu trúc tài liệu

Khoá gồm **tám cụm bài giảng + một bài tóm tắt toàn khoá + các tài nguyên tra cứu** (glossary, cross-reference, PDF index, exam checklist).

Cụm	Tên	Số bài	Trọng tâm
1	Giới thiệu Software Architecture	4	SA là gì, vì sao học, lộ trình
2	Design Principles (SOLID)	7	Cohesion/coupling + 5 nguyên lý SOLID

Cụm	Tên	Số bài	Trọng tâm
3	Architectural Thinking	4	Architecture vs Design, trade-off, modularity
4	Quality Attributes	4	NFR, architecture characteristics, ATAM, component-based
5	Fundamental Styles	5	Layered, Pipeline, Microkernel, Monolithic vs Distributed
6	Distributed Styles	5	Service-based, Microservices, Event-Driven, Space-Based
7	Documenting Architecture	4	Module/C&C/Allocation views, ADR
8	Case Studies	4	UAMS, Smart City, bài tập tổng hợp

Mỗi cụm bắt đầu bằng một bài **01-overview.md** giải thích tổng quan cụm: cụm này dạy gì, vì sao cần, kết nối ra sao với các cụm khác.

Cách đọc tài liệu

Có ba kiểu người đọc, mỗi kiểu nên đi một đường khác nhau:

Người đọc lần đầu (chuyên sâu)

Đọc theo thứ tự tự nhiên: Cụm 1 □ 2 □ 3 □ 4 □ 5 □ 6 □ 7 □ 8. Mỗi cụm khoảng 2-4 giờ đọc kỹ. Tổng cộng khoảng 30-40 giờ đọc + 10-20 giờ làm bài tập. Sau Cụm 4 bạn đã có nền tảng đủ vững để hiểu Cụm 5-6. Sau Cụm 6 bạn đủ trình để vẽ kiến trúc cho hệ thống cỡ trung. Sau Cụm 7-8 bạn có thể tự tin trình bày kiến trúc cho team.

Người ôn thi (nhANH)

Vào thẳng [Tóm tắt toàn khoá](#), đọc trong 1-2 giờ. Sau đó đọc [Exam Checklist](#) để biết các chủ đề thường được hỏi. Cuối cùng quay lại từng bài cụ thể nếu cần đào sâu.

Người tham chiếu nhanh (đã làm SA)

Vào thẳng cụm hoặc bài cần xem. Mỗi bài tự đứng đọc lập (có nhắc lại nền tảng cần thiết) nên bạn không cần đọc tuần tự. [Glossary](#) liệt kê alphabet các thuật ngữ với link tới bài chi tiết.

Triết lý biên soạn

Bốn nguyên tắc xuyên suốt:

1. **Trực giác trước hình thức.** Mỗi khái niệm mới đều bắt đầu bằng một câu hỏi đòi thường hoặc ví dụ ngắn, rồi mới định nghĩa chặt. Đọc bạn sẽ thấy nhịp điệu kiểu “Hãy tưởng tượng bạn đang xây...” trước khi gặp định nghĩa SOLID hay event sourcing.

2. **Ví dụ chạy được, không phải pseudo-code.** Code minh hoạ dùng Python, TypeScript, Java thật, không lý thuyết suông kiểu “giả sử class A...”. Bạn có thể copy chạy thử.
3. **Trade-off luôn rõ ràng.** Không có “best practice” tuyệt đối. Mỗi style/pattern/principle đều có *khi nào nên dùng, khi nào không, và cái giá bạn phải trả*. Một software architect giỏi không thuộc nhiều pattern hơn, họ chọn đúng pattern hơn.
4. **Tài liệu hoá là first-class.** Cụm 7 nguyên một cụm dành cho documenting vì kiến trúc không truyền đạt được là kiến trúc chỉ tồn tại trong đầu một người, và sẽ chết khi người đó rời team.

Quy ước hiển thị

- **Box xanh “Tóm tắt một dòng”:** ý cốt lõi của bài, đọc cái này nếu chỉ có 30 giây.
- **Box vàng “Sai lầm thường gặp”:** cảnh báo về cách hiểu sai phổ biến.
- **Code block:** ví dụ thực tế, đa số đã được kiểm tra cú pháp.
- **Mermaid diagram:** sơ đồ topology/flow, render được trực tiếp trên web.
- **“Khi nào dùng / khi nào không”:** bảng so sánh giúp quyết định nhanh.

Bắt đầu từ đâu?

Nếu bạn đọc lần đầu, click ngay vào [Cụm 1: Tổng quan](#) để bắt đầu. Nếu chưa chắc khoá này hợp với mình không, đọc thử [Bài 1.2: Software Architecture là gì?](#) để xem cách viết có khớp với gu của bạn không.

Nếu bạn đã biết Software Architecture và chỉ tra cứu, vào [Glossary](#) hoặc [Course Summary](#) để xem index.

Chúc bạn đọc vui và học hiệu quả.

1.1 Tổng quan Cụm 1: Giới thiệu Software Architecture

Tóm tắt một dòng: Software Architecture là tập hợp các quyết định khó-thay-đổi-nhất của một hệ phần mềm, và cụm này dạy bạn cách nhận ra những quyết định đó, đặt tên cho chúng, và bắt đầu cân nhắc chúng một cách có hệ thống.

Vì sao Cụm 1 cần đứng trước

Có một sai lầm rất phổ biến khi học Software Architecture: nhảy ngay vào học microservices, event-driven, hay DDD vì nghe có vẻ “kiến trúc”. Hậu quả là người học thuộc nhiều pattern nhưng không biết khi nào dùng, không biết vì sao pattern này lại sinh ra, và quan trọng nhất, không biết phân biệt vấn đề kiến trúc với vấn đề thiết kế thuần túy. Cụm 1 sửa sai lầm này bằng cách bắt đầu từ câu hỏi cơ bản nhất: “Software Architecture rốt cuộc là gì?”.

Sau khi đọc xong cụm này, bạn sẽ:

1. Phân biệt được Architecture với Design ở mức câu hỏi (Architecture trả lời “shape of system”, Design trả lời “shape of code inside a module”).
2. Hiểu vì sao những quyết định kiến trúc lại đắt khi sửa, và do đó vì sao chúng đáng để bỏ thời gian cân nhắc kỹ.
3. Có một lộ trình rõ ràng cho 7 cụm còn lại: cụm nào trả lời câu hỏi gì.
4. Biết cách dùng tài liệu này hiệu quả nhất theo nhu cầu (đọc lần đầu, ôn thi, hay tra cứu).

Bốn bài trong cụm

Cụm 1 ngắn nhất khoá, chỉ có 4 bài, vì mục tiêu là “khởi động” chứ không phải “đi sâu”. Đào sâu sẽ bắt đầu từ Cụm 2.

Bài 1.2: Software Architecture là gì?

Bài này định nghĩa Software Architecture qua ba góc nhìn bổ sung cho nhau: định nghĩa hình thức (theo Bass-Clements-Kazman), định nghĩa thực dụng (theo Martin Fowler), và định nghĩa “operational” (cái gì bạn phải làm khi bạn là một architect). Đặc biệt nhấn mạnh điểm: architecture không phải là “cái diagram đẹp”, mà là *tập hợp các quyết định khó sửa*.

Bài 1.3: Mục tiêu và kết quả học tập

Bài này nói rõ khoá học định cho bạn cái gì khi kết thúc: bạn sẽ biết tên gọi và bản chất 9 architecture styles phổ biến nhất, có thể đọc một bộ documentation kiến trúc bất kỳ và nhận ra đó là kiểu view nào, và đặc biệt là biết cách lập luận trade-off khi chọn style cho dự án mới. Bài cũng giải thích vì sao khoá không dạy *tất cả* (CQRS, Event Sourcing, Saga, hexagonal... đều bị bỏ ra) và bạn nên học gì tiếp theo.

Bài 1.4: Lộ trình đọc tài liệu

Bài này vẽ một sơ đồ phụ thuộc giữa các cụm, đề xuất 3 lộ trình khác nhau (chuyên sâu, ôn thi, tra cứu), và gợi ý thời gian ước lượng cho mỗi lộ trình. Đặc biệt có “shortcut paths” cho ai chỉ quan tâm tới một focus area cụ thể (vd: chỉ muốn học microservices, hoặc chỉ muốn ôn cho phỏng vấn).

Cách đọc Cụm 1

Cụm 1 dài tổng cộng chỉ khoảng 6000-8000 chữ. Đọc nghiêm túc mất khoảng 1-1.5 giờ. Đây là cụm bạn nên đọc *tuần tự* và *không skip*, vì nó set lên framework tư duy cho toàn khoá. Đặc biệt Bài 1.2 là bài quan trọng nhất, nếu bạn chỉ đọc một bài trong cả khoá, hãy đọc bài đó.

Sau khi đọc xong Cúm 1, bạn nên thấy một sự “click”: các thuật ngữ và khái niệm rời rạc bạn từng nghe sẽ bắt đầu có chỗ đứng trong một bản đồ chung. Nếu chưa thấy click, đừng vội đi tiếp, đọc lại Bài 1.2 và Bài 1.4 thêm một lần.

Kết nối với các cúm sau

Cúm 1 không có nội dung kỹ thuật, nó chỉ làm nhiệm vụ “đặt khung”. Toàn bộ kiến thức kỹ thuật bắt đầu từ Cúm 2:

- **Cúm 2 (SOLID)** trả lời câu hỏi: “Kiến trúc tốt bắt đầu từ đâu?”, câu trả lời là “từ code tốt”.
- **Cúm 3 (Architectural Thinking)** mở rộng từ code lên hệ thống: “Khi nào một quyết định trở thành kiến trúc?”.
- **Cúm 4 (Quality Attributes)** trả lời: “Tối ưu cái gì?”, vì bạn không thể tối ưu mọi thứ cùng lúc.
- **Cúm 5-6 (Architecture Styles)** dạy các “công cụ” chuẩn: 9 cách phổ biến để tổ chức một hệ thống.
- **Cúm 7 (Documenting)** dạy cách truyền đạt: kiến trúc không nói ra được là kiến trúc chết.
- **Cúm 8 (Case Studies)** áp dụng toàn bộ vào case thật.

Sai lầm cần tránh ngay từ đầu

Trước khi sang Bài 1.2, hãy ghi nhớ ba sai lầm phổ biến nhất khi tiếp cận Software Architecture:

1. **Coi architecture là “high-level design”**. Sai. Architecture không phải là design ở zoom level lớn hơn. Architecture là *một loại* quyết định khác, đặc trưng bởi tính khó-thay-đổi và ảnh hưởng tới quality attributes của toàn hệ. Bài 1.2 và Cúm 3 sẽ làm rõ điểm này.
2. **Coi architecture là “việc của architect”**. Sai. Trong một team trưởng thành, mọi engineer đều cần hiểu kiến trúc của hệ mình làm, chỉ là họ không tự quyết những thứ ở mức kiến trúc thôi. Bài 1.3 sẽ nói rõ.
3. **Coi architecture là cuộc đua đến microservices**. Sai cực kỳ phổ biến trong 5 năm gần đây. Microservices là *một style*, có chỗ dùng đúng và rất nhiều chỗ dùng sai. Phần lớn dự án bắt đầu nên là monolith (Cúm 5 sẽ giải thích). Đừng bị quyến rũ bởi pattern thời thượng, phải hiểu trade-off trước.

Nếu bạn đã thấy mình đang phạm một trong ba sai lầm trên, đừng lo. Hết Cúm 1 bạn sẽ gỡ được tất cả. Cùng bắt đầu với [Bài 1.2: Software Architecture là gì?](#).

1.2 Software Architecture là gì?

Tóm tắt một dòng: Software Architecture là tập hợp các quyết định khó-thay-đổi-nhất của một hệ phần mềm, đặc trưng bởi (a) tính bao trùm toàn hệ thống, (b) ảnh hưởng quyết định tới quality attributes, và (c) chi phí sửa cao tới mức phải cân nhắc rất kỹ ngay từ đầu.

Một câu hỏi đơn giản, ba câu trả lời khác nhau

Nếu bạn hỏi ba kỹ sư phần mềm “Software Architecture là gì?”, rất có thể bạn nhận được ba câu trả lời khác nhau và cả ba đều đúng một phần. Đó không phải lỗi của họ, đó là vì khái niệm này có nhiều lớp ý nghĩa, và mỗi cộng đồng (academia, industry, một dự án cụ thể) nhấn mạnh một lớp khác nhau. Bài này sẽ đi qua ba góc nhìn phổ biến nhất, sau đó tổng hợp lại thành một định nghĩa hành nghề được.

Góc nhìn 1: Định nghĩa hình thức (Bass-Clements-Kazman)

Trong sách kinh điển *Software Architecture in Practice* (xuất bản lần đầu 1998, đến nay đã ấn bản thứ tư), nhóm tác giả Bass-Clements-Kazman đưa ra định nghĩa sau:

“The software architecture of a system is the set of structures needed to reason about the system, which comprise software elements, relations among them, and properties of both.”

Diễn ra Tiếng Việt và bỏ formal-talk:

- Architecture là **một tập hợp các structure** (cấu trúc), không phải một structure duy nhất. Một hệ thống có nhiều cấu trúc song song: cấu trúc module, cấu trúc runtime, cấu trúc deployment...
- Mỗi structure gồm **elements** (thành phần) và **relations** (quan hệ giữa chúng).
- Cả elements và relations đều có **properties** (tính chất) đáng quan tâm.

Định nghĩa này tốt vì nó *bao quát*: bất cứ cái gì giúp bạn lý giải về hệ thống đều có thể coi là một phần của architecture. Nhưng nó hơi mơ hồ cho người mới: “structure nào quan trọng” thì câu này không trả lời.

Góc nhìn 2: Định nghĩa thực dụng (Martin Fowler)

Trong bài *Who Needs an Architect?* (2003), Martin Fowler, một trong những voice ảnh hưởng nhất ngành Software, phỏng vấn nhiều architect đầu ngành và rút ra:

“Architecture is about the important stuff. Whatever that is.”

Câu này nghe đùa nhưng cực kỳ sâu. Ý của Fowler:

- Architecture không phải là một loại đối tượng cụ thể (không phải “diagram”, không phải “module list”, không phải “API spec”).
- Architecture là **whatever decisions are hard to change later**. Cái gì khó sửa, cái đó là kiến trúc. Cái gì dễ sửa, cái đó là thiết kế chi tiết hoặc implementation.
- “Important” ở đây không phải “to bigger” hay “more abstract”, mà là **important to the long-term success** của hệ thống.

Định nghĩa này thực dụng hơn nhiều, và là cách phần lớn architect industry hiểu khái niệm này. Nhưng nó vẫn cần thêm tiêu chí cụ thể để áp dụng.

Góc nhìn 3: Định nghĩa operational (cái gì architect thực sự làm)

Nếu nhìn vào công việc hàng ngày của một software architect ở công ty trưởng thành, bạn sẽ thấy họ thực sự dành thời gian cho:

1. **Lựa chọn architecture style cho hệ mới:** monolith hay microservices? Layered hay event-driven? Style nào phù hợp với quality attributes ưu tiên?
2. **Định nghĩa boundary giữa các component:** cái gì là module riêng, cái gì là sub-module, cái gì là phần của một module lớn hơn?
3. **Quyết định technology stack chiến lược:** Java vs Go vs Node? PostgreSQL vs MongoDB? Kafka hay RabbitMQ?
4. **Trade-off khi xung đột quality attributes:** chọn performance hay maintainability? Chọn consistency hay availability?
5. **Documenting và truyền đạt:** viết Architecture Decision Records (ADR), vẽ diagrams chuẩn, review design của team.
6. **Cross-team coordination:** đảm bảo các team build các phần khác nhau của hệ vẫn ghép được với nhau.

Tất cả 6 việc trên đều có một đặc điểm chung: **quyết định một lần ảnh hưởng dài hạn, sửa lại thì rất đắt**. Đó chính là essence của architecture.

Tổng hợp: Định nghĩa “operational” của tài liệu này

Trong toàn bộ khoá học, chúng ta sẽ dùng định nghĩa sau:

Software Architecture là tập hợp các quyết định thiết kế của một hệ phần mềm thoả mãn cả ba tiêu chí: 1. **Bao trùm toàn hệ thống** (system-wide), không thuộc về một module riêng lẻ. 2. **Ảnh hưởng quyết định tới quality attributes** (performance, scalability, security, maintainability...) thay vì chỉ tới functional behavior. 3. **Chi phí thay đổi cao**, sửa sau khi đã code/deploy rất tốn kém.

Bất cứ quyết định nào đạt cả ba tiêu chí trên là kiến trúc. Cái gì không đạt là design thông thường hoặc implementation detail.

Ba ví dụ minh họa

Để định nghĩa trên trở nên cụ thể, hãy xét ba quyết định và xác định cái nào là architectural, cái nào không.

Ví dụ 1: Chọn dùng PostgreSQL hay MongoDB cho hệ thương mại điện tử

- Bao trùm toàn hệ? ☐ (mọi service đều dùng tới database này).
- Ảnh hưởng quality attribute? ☐ (consistency, query expressiveness, scalability đều khác nhau giữa hai lựa chọn).
- Chi phí thay đổi cao? ☐ (migrate database sau 2 năm production là một dự án 6-12 tháng).

☐ **Architectural decision.** Phải được cân nhắc kỹ ngay đầu dự án.

Ví dụ 2: Đặt tên biến userCount hay numUsers trong một function

- Bao trùm toàn hệ? ☐ (chỉ trong scope một function).
- Ảnh hưởng quality attribute? ☐ (cùng lắm là code style).
- Chi phí thay đổi? ☐ (IDE rename, một phím tắt).

□ **Implementation detail.** Đừng tranh cãi tới 30 phút trong code review.

Ví dụ 3: Dùng JWT hay session cookie cho authentication trong một SaaS B2B

- Bao trùm toàn hệ? □ (mọi protected endpoint đều phải verify).
- Ảnh hưởng quality attribute? □ (scalability, security, sessions revocation đều khác).
- Chi phí thay đổi? □ (medium, có thể migrate dần qua dual support nhưng vẫn cần effort).

□ **Architectural** (tier 2). Cần thảo luận team trước khi chốt.

Bài tập nhỏ cho bạn: đặt 5 quyết định gần đây trong dự án bạn đang làm vào framework 3 tiêu chí này. Bạn sẽ thấy phần lớn quyết định *không* là architectural (mặc dù cãi nhau cũng nhiều). Chỉ một số nhỏ thực sự deserve mức attention architectural.

“Architecture is design”, nhưng không phải mọi design đều architectural

Có một câu nổi tiếng của Grady Booch:

“All architecture is design, but not all design is architecture.”

Hiểu về đầu: architecture cũng là một dạng design, không có gì huyền bí. Nó vẫn là quyết định “structure this way vs that way”. Hiểu về sau: nhưng chỉ một tập con của design quyết định mới đáng được gọi là architecture, tập con đạt ba tiêu chí ở trên.

Hệ quả: ranh giới giữa architecture và design *không cố định*, nó phụ thuộc context. Cùng một quyết định có thể là architectural trong project A nhưng là implementation detail trong project B. Ví dụ:

- Trong một startup 5 người làm MVP: cấu trúc class hierarchy của một module là implementation detail (vì cả team thuộc, sửa nhanh).
- Trong một enterprise 200 engineers: cùng quyết định đó có thể là architectural (vì 50 người sẽ depend, sửa cần coordinate).

Điều này dẫn tới một insight quan trọng: **vai trò architect không phải để “ra quyết định đúng” mà là “nhận ra quyết định nào quan trọng đến mức cần được xem xét nghiêm túc”.**

Vì sao kiến trúc lại đắt khi sửa?

Có ba lý do chính khiến architectural decisions đắt khi sửa, và hiểu rõ ba lý do này giúp bạn cẩn thận hơn ngay từ đầu:

1. Cascade effect

Một quyết định architectural thường được nhiều phần khác depend vào. Đổi database từ MongoDB sang PostgreSQL không chỉ là rewrite data access layer, bạn còn phải đổi schema design, query patterns, ORM, deployment, monitoring, backup strategy, on-call runbook... Mỗi cái lại depend vào hàng tá thứ khác. Cascade này có thể mất hàng tháng untangle.

2. Conway's law

Architecture của hệ phản chiếu organization structure của team build nó. Một khi team đã được tổ chức xung quanh một architecture nhất định (vd: 5 team mỗi team own một microservice), đổi architecture thường đồng nghĩa đổi org chart, việc cực kỳ tốn về mặt chính trị và con người. Conway's law sẽ được nhắc lại nhiều lần trong khoá.

3. Sunk cost của technology lock-in

Mỗi technology choice tích lũy “muscle memory” của team: tools, libraries, internal frameworks, training, hiring criteria... Đổi tech stack đồng nghĩa team phải học lại, viết lại tooling, có khi tuyển người mới. Sunk cost này tăng tuyến tính với tuổi project.

Một “architect” thực sự làm gì?

Tài liệu này tránh dùng từ “architect” như một chức danh, vì:

- Ở một số công ty, architect là chức danh chính thức (vd: Solution Architect, Enterprise Architect).
- Ở công ty khác, không có chức danh này, quyết định kiến trúc được chia cho Tech Lead, Staff Engineer, hoặc CTO.
- Ở startup, một developer 5 năm kinh nghiệm có thể đã đang làm việc của một architect mà không gọi tên thế.

Vai trò “architect”, bất kể chức danh, bao gồm:

1. **Phân biệt architectural decision với design decision**: dành thời gian cho cái thứ nhất, để cái thứ hai cho engineer thực thi tự quyết.
2. **Cân nhắc trade-off đa chiều**: không bao giờ có “best architecture”, chỉ có “best for current context”.
3. **Truyền đạt và document**: để team hiểu *vì sao* quyết định kia, không chỉ *cái gì* được quyết.
4. **Sẵn sàng review và adjust**: kiến trúc evolve, không có “set and forget”.
5. **Code đủ để hiểu pain**: architect xa rời code lâu sẽ ra quyết định lý thuyết, không thực dụng. Cụm 3 sẽ nói rõ.

Tóm tắt

- Software Architecture là tập hợp **quyết định khó-sửa-nhất** ảnh hưởng toàn hệ và quality attributes.
- Có ba định nghĩa phổ biến (Bass-Clements-Kazman, Fowler, operational), và cả ba bổ sung nhau.
- Phân biệt architectural decisions với implementation details bằng **3 tiêu chí**: bao trùm, ảnh hưởng QA, đắt khi sửa.
- Architecture đắt khi sửa do **cascade effect**, **Conway’s law**, và **technology lock-in**.
- “Architect” là một *vai trò*, không phải bắt buộc một *chức danh*.

Bài tiếp theo: [Mục tiêu và kết quả học tập](#), khoá này định cho bạn cái gì khi kết thúc.

1.3 Mục tiêu và kết quả học tập

Tóm tắt một dòng: Khoá này muốn bạn ra trường biết tên 9 architecture styles phổ biến, đọc được documentation kiến trúc bất kỳ, và lập luận được trade-off khi chọn architecture cho dự án mới, chứ không cố làm bạn thành expert ở mọi pattern.

Vì sao cần nói rõ mục tiêu?

Một sai lầm phổ biến khi học SA là cố học “tất cả”: SOLID, DDD, CQRS, Event Sourcing, Saga pattern, Outbox pattern, hexagonal, clean architecture, onion architecture, BFF, sidecar, service mesh, mesh app... Mỗi chủ đề lại dẫn tới 5 chủ đề khác. Không ai có thể master tất cả, và cố làm thế chỉ dẫn tới sự nông cạn ở mọi chỗ.

Khoá này có scope rõ ràng và *cố tình* không dạy mọi thứ. Đọc kỹ phần “Không nằm trong scope” để biết bạn cần học gì sau khoá.

Mục tiêu khoá (Aims)

Sau khi hoàn thành khoá, bạn sẽ:

Mục tiêu 1: Nhận biết và lập luận về architectural decisions

Bạn sẽ phân biệt được:

- Architectural decision vs Design decision vs Implementation detail (Bài 1.2 và Cụm 3 build kỹ).
- Architectural characteristic (cái bạn tối ưu) vs Architectural style (cách bạn tổ chức để tối ưu).
- Trade-off đa chiều: vì sao “best architecture” không tồn tại trong vacuum.

Mục tiêu 2: Áp dụng SOLID và các nguyên lý thiết kế cấp module

Cụm 2 dạy bạn 5 nguyên lý SOLID kèm cohesion-coupling. Sau cụm này, bạn:

- Nhận ra anti-pattern vi phạm SOLID trong code thực.
- Refactor được code vi phạm SRP/OCP/LSP/ISP/DIP về dạng tuân thủ.
- Hiểu được khi nào áp dụng SOLID là *over-engineering* (vì SOLID không miễn phí).

Mục tiêu 3: Biết 9 architecture styles phổ biến

Cụm 5-6 đi qua 9 style chính: Layered, Pipeline, Microkernel, Monolithic (đối chiếu), Service-based, Microservices, Event-Driven, Space-Based, và một số biến thể. Với mỗi style bạn sẽ biết:

- Topology cốt lõi (vẽ được trên giấy).
- Quality attributes mà nó tối ưu (vd: Microservices tối ưu scalability/deployability, sacrificing simplicity).
- Khi nào dùng, khi nào không.
- Cách evolve giữa các style (vd: monolith $\square$ service-based $\square$ microservices).

Mục tiêu 4: Đọc và viết documentation kiến trúc chuẩn

Cụm 7 dạy 3 loại view (Module, Component-and-Connector, Allocation), đây là chuẩn de-facto của ngành. Sau cụm này bạn:

- Đọc được architecture document bất kỳ và nhận ra đó là loại view nào.

- Viết được architecture document cho hệ mình thiết kế, dùng đúng notation chuẩn.
- Biết khi nào dùng ADR (Architecture Decision Record) và viết được một ADR đúng cách.

Mục tiêu 5: Áp dụng vào case study thực

Cụm 8 đi qua 2 case study chi tiết (UAMS, Academic Management System, Smart City Traffic Detection) và một bộ bài tập tổng hợp. Sau cụm này bạn:

- Tự thiết kế kiến trúc end-to-end cho hệ cỡ trung (50-500k user) dựa trên requirement.
- Trình bày được kiến trúc đó cho người không-kỹ-thuật hiểu (vd: PM, business stakeholder).
- Defend được lựa chọn trước câu hỏi “vì sao không dùng X?”.

Kết quả học tập (Learning Outcomes)

Kết quả học tập là phiên bản đo được của mục tiêu. Sau khi hoàn thành khoá, bạn có thể:

Mã	Kết quả	Cụm dạy
LO1	Định nghĩa SA và phân biệt architectural vs design decisions	1, 3
LO2	Áp dụng 5 nguyên lý SOLID khi review/refactor code	2
LO3	Phân tích trade-off khi chọn architecture cho ứng dụng cho trước	3, 4
LO4	Xác định 5-10 architecture characteristics ưu tiên cho hệ mới	4
LO5	Mô tả topology và trade-off của 9 architecture styles phổ biến	5, 6
LO6	Chọn architecture style phù hợp cho một dự án dựa trên QA priorities	5, 6, 8
LO7	Vẽ Module View, C&C View, Allocation View cho hệ thiết kế	7
LO8	Viết Architecture Decision Record (ADR) cho một quyết định cụ thể	7
LO9	Thiết kế end-to-end kiến trúc cho hệ cỡ trung trong 2-3 giờ	8
LO10	Đọc, đánh giá, và phản biện architecture của hệ có sẵn	All

Bộ LO này khá tham vọng. Đạt được toàn bộ đòi hỏi 30-50 giờ học nghiêm túc + 10-20 giờ thực hành. Đừng vội nản nếu sau lần đọc đầu tiên chưa đạt, kiến trúc là kỹ năng tích lũy.

Không nằm trong scope

Khoá này cố tình bỏ ra ngoài các chủ đề sau (để giữ scope khả thi):

1. Domain-Driven Design (DDD)

DDD là một methodology để mô hình hoá domain phức tạp. Nó liên quan tới SA nhưng là một body of knowledge riêng (sách của Eric Evans ~500 trang). Học sau khoá này: đọc *Domain-Driven Design Distilled* (Vaughn Vernon) cho phiên bản gọn, hoặc *Implementing Domain-Driven Design* (Vaughn Vernon) cho phiên bản đầy đủ.

2. CQRS, Event Sourcing, Saga, Outbox pattern

Đây là các pattern phổ biến trong distributed systems hiện đại nhưng thuộc về *pattern level*, không phải *style level*. Khoá này dừng ở style. Học sau: *Microservices Patterns* (Chris Richardson) là tài liệu tốt nhất cho các pattern này.

3. Hexagonal / Clean / Onion Architecture

Ba “kiến trúc” này thực ra là cùng một idea (separation of business logic khỏi infrastructure) đóng gói khác nhau. Quan trọng nhưng overlap nhiều với SOLID + layered architecture. Học sau: chương 22-24 của *Clean Architecture* (Robert C. Martin).

4. Cloud-native specifics

Service mesh (Istio, Linkerd), serverless (Lambda, Cloud Run), container orchestration (Kubernetes), API gateway... đều là các topic con của cloud architecture. Khoá này chỉ nhắc đến khi cần trong Cụm 6. Học sau: *Cloud Native Patterns* (Cornelia Davis) hoặc tài liệu chính thống của cloud provider.

5. Performance engineering chi tiết

Khoá có nói về performance là một quality attribute, nhưng không đi sâu vào: caching strategies, database tuning, load testing methodology, profiling tools. Học sau: *Designing Data-Intensive Applications* (Martin Kleppmann), sách gối đầu giường về scaling.

6. Security architecture chi tiết

Tương tự, security được nhắc đến nhưng không sâu. Nếu quan tâm security thực sự, học một khoá riêng hoặc xem qua [Software Security Foundation](#), sister course của khoá này.

Lộ trình học tiếp sau khoá

Sau khi hoàn thành khoá này, đây là lộ trình gợi ý theo focus area:

Nếu bạn đi hướng Architect chuyên nghiệp

1. Đọc *Fundamentals of Software Architecture* (Mark Richards, Neal Ford), sách chính của ngành.
2. Đọc *Software Architecture: The Hard Parts* (Neal Ford, Mark Richards), phần distributed deeply.
3. Học DDD qua *Domain-Driven Design Distilled* (Vaughn Vernon).
4. Bắt đầu viết ADR thật trong dự án của bạn.

Nếu bạn đi hướng Tech Lead

1. Đọc *The Software Architect Elevator* (Gregor Hohpe), về vai trò architect trong tổ chức.
2. Đọc *Team Topologies* (Skelton, Pais), về cách tổ chức team xung quanh architecture.
3. Practice trade-off analysis trên dự án thật của team mình.

Nếu bạn đi hướng Distributed Systems

1. Đọc *Designing Data-Intensive Applications* (Martin Kleppmann), bible của ngành.
2. Học Kafka, gRPC, distributed consensus (Raft, Paxos).
3. Practice với một hệ thật (vd: build một service mesh nhỏ).

Nếu bạn đi hướng Cloud Architect

1. Lấy chứng chỉ AWS Solutions Architect Associate hoặc Google Professional Cloud Architect.
2. Đọc *Cloud Native Patterns* (Cornelia Davis).
3. Practice với Kubernetes thật + ít nhất một managed service mesh.

Tóm tắt

- Khoá có 5 mục tiêu chính, đo được qua 10 learning outcomes.
- Khoá *cố tình* không dạyDDD, CQRS, hexagonal, cloud-native deeply, performance/security deeply.
- Sau khoá, có 4 lộ trình tiếp theo tùy focus area.
- Đạt được toàn bộ LO đòi hỏi 30-50 giờ học + thực hành.

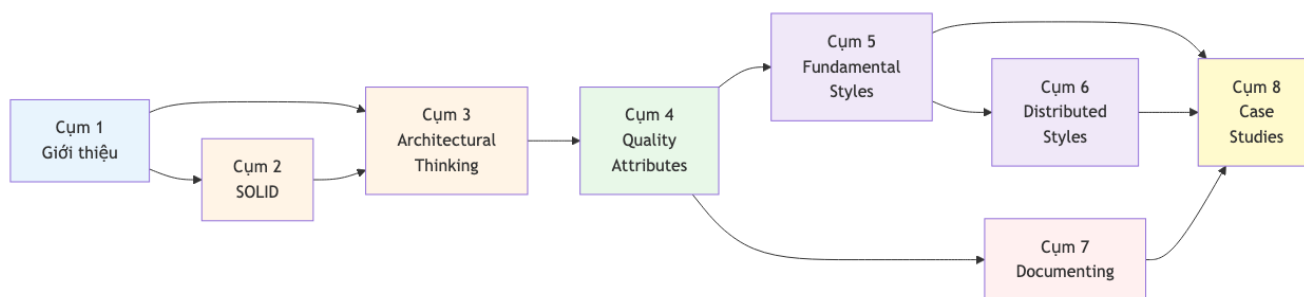
Bài tiếp: [Lộ trình đọc tài liệu](#), chọn cách đọc phù hợp với bạn.

1.4 Lộ trình đọc tài liệu

Tóm tắt một dòng: Có ba lộ trình tùy vào mục đích (chuyên sâu 30-40h, ôn thi 5-8h, tra cứu nhanh on-demand), và 4 shortcut paths cho ai chỉ quan tâm một focus area (microservices, documenting, design principles, hoặc preparation for senior interview).

Sơ đồ phụ thuộc giữa các cụm

Trước khi chọn lộ trình, hãy nhìn xem các cụm phụ thuộc nhau ra sao:



Hình 1: Sơ đồ 1

Quy ước màu:

- **Xanh nhạt:** warm-up (đọc nhẹ).
- **Vàng cam:** design level (SOLID + thinking).
- **Xanh lá:** characteristics (NFR + ATAM).
- **Tím:** structural styles.
- **Hồng:** communication (documenting).
- **Vàng:** application (case studies).

Ba lộ trình chính

Lộ trình A: Chuyên sâu (chuẩn, 30-50 giờ)

Đối tượng: học viên cao học cần điểm cao, lập trình viên chuẩn bị thăng tiến lên Tech Lead/Architect.

Tuần	Cụm	Thời gian	Mục tiêu
1	Cụm 1	1-2h	Set up framework tư duy
1-2	Cụm 2	6-10h	Master SOLID, refactor 5-10 đoạn code thật
3	Cụm 3	4-6h	Trade-off thinking, distinguish arch vs design
4	Cụm 4	4-6h	Identify QA cho 2-3 hệ thực tế bạn đang dùng
5	Cụm 5	5-7h	Vẽ topology 4 fundamental styles
6	Cụm 6	5-7h	Vẽ topology 4 distributed styles, so sánh
7	Cụm 7	4-6h	Vẽ 3 view cho hệ bạn đang làm, viết 2 ADR
8	Cụm 8	6-10h	Làm cả 2 case study + bộ exercise
9	Course Summary	1-2h	Tổng hợp, ôn lại weak spots

Tổng: ~35-55 giờ trong 8-9 tuần (4-7h/tuần). Khuyến khích đi cùng một dự án thật để áp dụng từng cụm.

Lộ trình B: Ôn thi (nhANH, 5-8 giờ)

Đối tượng: đã có background SA, cần ôn nhanh cho thi cử hoặc phỏng vấn.

Thứ tự	Tài liệu	Thời gian
1	Course Summary	1-2h
2	Exam Checklist	30 phút
3	Cụm 2 (SOLID), chỉ overview + 5 SOLID files	1.5h
4	Cụm 5+6, chỉ overview + 1-2 style chính	1.5h
5	Cụm 7, chỉ overview + 4+1 model nhanh	30 phút
6	Glossary , scan để spot weak terms	1h

Tổng: ~5-8 giờ. Hiệu quả nếu bạn đã quen với khái niệm và chỉ cần refresh.

Lộ trình C: Tra cứu (on-demand)

Đối tượng: đã làm SA, vào tra cứu nhanh khi cần.

- Vào [Glossary](#) để tìm thuật ngữ □ click vào link bài chi tiết.
- Vào [Cross-reference](#) để xem bản đồ chủ đề.
- Vào bài cụ thể bất kỳ, mỗi bài được viết tự-độc-lập với prerequisites được nhắc lại ngắn gọn.

Không có thời gian cố định. Phù hợp khi cần check một concept hoặc compare 2-3 styles.

Bốn shortcut paths theo focus area

Nếu bạn chỉ quan tâm một focus area cụ thể, đây là sequence tối ưu:

Shortcut 1: “Tôi chỉ muốn hiểu microservices” (8-12 giờ)

1. [Cụm 1.2: SA là gì?](#), 30 phút.
2. [Cụm 3: Architectural Thinking](#), 4-6h.
3. [Cụm 4: Quality Attributes](#), chỉ overview + identifying. 2h.
4. [Cụm 5.2: Monolithic vs Distributed](#), 1h.
5. [Cụm 6.3: Microservices](#) + 6.4 Event-Driven, 2-3h.

Shortcut 2: “Tôi muốn dạy team cách document architecture” (4-6 giờ)

1. [Cụm 1.2: SA là gì?](#), 30 phút.
2. [Cụm 4.3: Identifying characteristics](#), 1h.
3. Cả [Cụm 7: Documenting](#), 4-6h.
4. Practice viết ADR cho 1 quyết định gần nhất trong team, 1h.

Shortcut 3: “Tôi muốn master SOLID và improve code quality team” (10-15 giờ)

1. [Cụm 1.2: SA là gì?](#), 30 phút.
2. Cả [Cụm 2: Design Principles](#), 6-10h.
3. [Cụm 3.4: Modularity](#), 1.5h.
4. Practice refactor 5-10 đoạn code thật vi phạm SOLID, 3-5h.

Shortcut 4: “Tôi chuẩn bị interview Senior/Staff Engineer” (15-20 giờ)

1. Cụm 1 đầy đủ, 1-2h.
2. Cụm 2 SOLID, 4-6h.
3. Cụm 3 + 4, 6-8h (quan trọng cho behavior-level questions).
4. Cụm 5 + 6, 4-6h (system design questions thường hỏi style trade-off).
5. Cụm 7.1 + 7.2, 1.5h.
6. [Course Summary](#), 1-2h.

Lưu ý khi học**Đừng “cày” lý thuyết một mình**

SA là kỹ năng *applied*. Đọc lý thuyết suông sẽ quên rất nhanh. Mỗi cụm hãy:

- Áp dụng vào ít nhất một dự án bạn đang làm hoặc đã làm.
- Vẽ topology trên giấy (không gõ vào tool).
- Giải thích lại cho đồng nghiệp/bạn cùng học, nếu giải thích không trôi chảy là bạn chưa hiểu.

Đừng ngại quay lại

Đọc lần đầu xong Cụm 6 mà chưa hiểu thật sự là chuyện bình thường. Quay lại Cụm 4 (quality attributes) thường giúp click. SA có nhiều khái niệm circular, phải đi qua vài lần để thấm.

Đừng bị quyến rũ bởi trends

Trong 5 năm gần đây, microservices, event-driven, serverless được hype rất nhiều. Khóa này dạy bạn để *phản biện* được trends, không phải để follow chúng. Một monolith được thiết kế tốt có thể serve 80% dự án tốt hơn microservices được thiết kế sai.

Code thực sự là phần thiết yếu

Bài 3.3 (Balancing Architecture and Hands-On Coding) sẽ nói rõ vì sao architect cần vẫn code. Tuyệt đối đừng coi SA là “thoát code lên design”. Quan điểm đó dẫn tới kiến trúc bị disconnect với thực tế.

Tiếp theo

Hết Cụm 1. Bạn nên bắt đầu Cụm 2 với [Tổng quan Design Principles](#).

Nếu muốn xem bản đồ toàn khóa ở mức gọn hơn, vào [Course Summary](#). Nếu cần check thuật ngữ, vào [Glossary](#).